

Python Scripting

NCSC101

Prof. Pranay Kumar Saha

October 8, 2025





What is Python Scripting?

- Writing Python programs to automate tasks
- Typically shorter programs focused on specific tasks
- Common uses:
 - File manipulation
 - System administration
 - Data processing
 - Web scraping
 - Task automation
- Scripts vs. Programs: Scripts are usually simpler, task-specific



Script Structure

```
import os
import sys

MAX_SIZE = 1000

def main():
    """Main function"""
    print("Script execution")

if __name__ == "__main__":
    main()
```



What is the sys module?

The 'sys' module is a fundamental part of Python's standard library.

- It provides access to variables and functions that interact strongly with the Python **interpreter** and its environment.
- Think of it as a bridge between your Python script and the system it's running on.
- You must import it to use it: **import sys**



Python Script ↔ sys Module ↔ System



sys.argv: Command-Line Arguments

`sys.argv` is a **list** containing the command-line arguments passed to your script.

- The first item, `sys.argv[0]`, is always the name of the script itself.
- The following items are the arguments you provide when running the script.

Example Code: `show_args.py`

```
import sys

print(f"The script name is: {sys.argv[0]}")
print(f"The arguments are: {sys.argv[1:]}")
```



What's the Output?

Consider this script, named `arg_quiz.py`:

```
import sys

script_name = sys.argv[0]
first_arg = sys.argv[1]
all_args = sys.argv

print(f"Length is: {len(all_args)}")
print(f"First argument is: {first_arg}")
```

If you run it from the terminal like this:

```
$ python arg_quiz.py hello world
```



sys.path: The Module Search Path

`sys.path` is a **list** of strings that specifies the search directories for modules.

- When you write `import my_module`, Python searches for `my_module.py` in each directory listed in `sys.path`.
- The first directory is usually the location of the script you are running.

Example Code

```
import sys

# Print all directories in the module search path
for path in sys.path:
    print(path)
```



sys.platform and sys.version

sys.platform

A string identifying the operating system.

- 'win32' for Windows
- 'linux' for Linux
- 'darwin' for macOS

sys.version

A string with details about the Python interpreter version.

```
import sys

if sys.platform == "win32":
    print("You are running Windows!")
else:
    print(f"You are on {sys.platform}.")

print(f"Python Version: {sys.version.split()[0]}")
```



`sys.exit()`: Exiting the Script

The '`sys.exit()`' function allows you to terminate your script's execution immediately.

- You can optionally pass an **exit code** (an integer).
- An exit code of **0** conventionally means success.
- Any non-zero exit code (e.g., **1**) signals that an error occurred.



sys.exit(): Exiting the Script

Example Code

```
import sys

age = 17

if age < 18:
    print("Access denied: You are too young.")
    sys.exit(1) # Exit with an error code

print("Welcome! Access granted.") # This line will not run
```



Quiz: What's the Output?

Consider this script, named `check_user.py`:

```
import sys

if len(sys.argv) < 2:
    print("Error: Please provide a username.")
    sys.exit(1)

username = sys.argv[1]
print(f"Welcome, {username}!")
print("Script finished.")
```

What is the output if we run it like this? `$ python check_user.py`



Standard Streams: `stdin`, `stdout`, `stderr`

The `'sys'` module gives you access to the three standard I/O streams:

- **`sys.stdin`**: Standard Input (usually the keyboard)
- **`sys.stdout`**: Standard Output (where `'print()'` sends text)
- **`sys.stderr`**: Standard Error (for error messages)

```
import sys

# print() is a wrapper around sys.stdout.write()
sys.stdout.write("This is a standard message.\n")

# It's good practice to send errors to stderr
sys.stderr.write("This is an error message.\n")
```



Summary

We've covered the essential parts of the 'sys' module:

- **sys.argv**: Access command-line arguments.
- **sys.path**: See where Python looks for modules.
- **sys.platform**: Check the operating system.
- **sys.version**: Get the Python interpreter version.
- **sys.exit()**: Stop the script with an exit code.
- **sys.stdout** / **stderr**: Write to output and error streams.

The 'sys' module is your tool for building powerful command-line applications and scripts that can adapt to their environment.



Why build command-line tools?

- Automate tasks and batch workflows
- Scriptable, composable, easy to integrate in CI
- Portable: runs anywhere Python runs
- Clear contracts via `--help` and exit codes



Meet argparse

- Standard library module for parsing command-line options
- Generates **--help** and usage for free
- Validates types, required args, defaults, choices, etc.



What is the argparse module?

While `sys.argv` gives you a raw list of arguments, `argparse` is a powerful library for creating robust command-line interfaces (CLIs).

`sys.argv`

- A simple list of strings.
- You must parse it manually.
- No help messages.
- Error handling is manual.

`argparse`

- Parses arguments for you.
- **Automatically generates help** and usage messages (`-h`, `--help`).
- Handles different argument types.
- Issues clear errors for invalid input.



The Three Core Steps of `argparse`

Using `argparse` always follows the same simple pattern:

1. **Create a parser object** from the `ArgumentParser` class.
2. **Add arguments** to the parser using the `.add_argument()` method.
3. **Parse the arguments** from the command line using `.parse_args()`.



The Three Core Steps of argparse

```
import argparse
# 1. Create the parser
parser = argparse.ArgumentParser(description="A simple demo.")

# 2. Add arguments (we will add some here)
# parser.add_argument(...)

# 3. Parse the command-line arguments
args = parser.parse_args()
```



Adding a Positional Argument

A **positional argument** is required and its meaning is determined by its position on the command line. It does not start with a dash ('-').

Example Script: greet.py

```
import argparse

parser = argparse.ArgumentParser()

# Add a positional argument named 'name'
parser.add_argument("name", help="The name of the person to
    greet")
args = parser.parse_args()
print(f"Hello, {args.name}!")
```



Quiz: What's the Output?

Given the script `greet.py` from the previous slide:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("name", help="The name of the person to
    greet")
args = parser.parse_args()
print(f"Hello, {args.name}!")
```

If you run it from the terminal like this:

```
$ python greet.py Alice
```



The Magic of `--help`

One of the best features of 'argparse' is that it automatically generates a detailed help message for you.

If you run our `greet.py` script with the `-h` or `--help` flag:

```
$ python greet.py --help
```

```
usage: greet.py [-h] name
```

```
positional arguments:
```

```
  name                The name of the person to greet
```

```
optional arguments:
```

```
  -h, --help          show this help message and exit
```



Adding Optional Arguments (Flags)

An **optional argument** (or flag) starts with - (short name) or -- (long name). It is not required.

The `action="store_true"` makes the argument a boolean flag. If present, its value is `True`; otherwise, it's `False`.



Adding Optional Arguments (Flags)

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("-v", "--verbose", help="Increase output
    verbosity"
)
args = parser.parse_args()

if args.verbose:
    print("Verbosity turned on")
else:
    print("Verbosity turned off")
```

What happens if you run this script with `$ python script.py --verbose`? What if you run it with no arguments?



Adding action="store_true" (Flags)

Example: Adding a --verbose flag

```
import argparse
parser = argparse.ArgumentParser()

# Add an optional argument (a flag)
parser.add_argument("-v", "--verbose",
                    action="store_true",
                    help="Enable verbose output")

args = parser.parse_args()
if args.verbose:
    print("Verbose mode is ON.")
else:
    print("Verbose mode is OFF.")
```



Quiz: What's the Output?

Consider this script, named `process.py`:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("filename", help="File to process")
parser.add_argument("-q", "--quiet", action="store_true", help="
    Suppress output")
args = parser.parse_args()

if not args.quiet:
    print(f"Processing file: {args.filename}")
```

```
$ python process.py report.txt --quiet
```



Counting Occurrences with action="count"

This action counts how many times an optional argument appears.

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument(
    "-v", "--verbose",
    action="count",
    default=0,
    help="Increase verbosity level
    (-v, -vv, -vvv).")
)
args = parser.parse_args()
print(f"Verbosity level: {args.
    verbose}")
```

Example Runs

```
$ python script.py
Output: Verbosity level: 0
$ python script.py -v
Output: Verbosity level: 1
$ python script.py -vvv
Output: Verbosity level: 3
```



Building Lists with action="append"

This action appends the argument's value to a list. It can be used multiple times.

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument(
    '--item',
    action='append',
    help='Add an item to the list.'
)
args = parser.parse_args()

print(f"Items collected: {args.item
    }")
```

Example Runs

```
$ python script.py --item
apple --item banana
Output: Items collected:
['apple', 'banana'] $
```

```
python script.py
Output: Items collected:
None
```



Specifying Argument Types

By default, all arguments are read as strings. You can use the `'type'` parameter to automatically convert the value to another type, like an integer or float.

`'argparse'` will even produce a user-friendly error if the input can't be converted!



Specifying Argument Types

Example: A script to multiply a number

```
import argparse
parser = argparse.ArgumentParser()

parser.add_argument("number", type=int, help="A number to
    multiply")
parser.add_argument("--factor", type=int, default=2, help="
    Multiplication factor")

args = parser.parse_args()

result = args.number * args.factor
print(f"The result is: {result}")
```



Setting Default Values with `default`

The `default` keyword specifies a value to be used if an optional argument is not provided.

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument(
    "--level",
    type=int,
    default=1,
    help="Set the difficulty level (default: 1)")
args = parser.parse_args()

print(f"Difficulty set to level {args.level}")
```

```
$ python script.py
$ python script.py
--level 5
```



Putting It All Together

Given this script, named `repeat.py`:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("message", help="Message to repeat")
parser.add_argument("-t", "--times", type=int, default=3, help="
    Number of repetitions")
args = parser.parse_args()

for _ in range(args.times):
    print(args.message)
```

```
$ python repeat.py "Go Team!" --times 2
```



Summary

‘argparse’ is the standard way to create powerful command-line interfaces.

- **ArgumentParser()**: Creates the parser.
- **.add_argument()**: Defines your arguments.
 - **Positional**: Required arguments (e.g., “filename”).
 - **Optional**: Flags that start with ‘-’ or ‘--’ (e.g., “--verbose”).
- **.parse_args()**: Does the magic of parsing ‘sys.argv’.
- Key parameters include ‘help’, ‘type’, ‘action’, and ‘default’.
- You get robust error handling and help messages for free!



Minimal argparse script

```
import argparse

parser = argparse.ArgumentParser(
    prog="greeter",
    description="Greet someone a number of times",
)
parser.add_argument("name", help="who to greet")
parser.add_argument("count", type=int, help="how many times")
args = parser.parse_args()

for i in range(args.count):
    print(f"Hello, {args.name}! ({i+1})")
```



Limiting Choices with choices

You can restrict an argument's value to a specific set of choices.

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument(
    "--mode",
    choices=['easy', 'medium', 'hard'],
    default='easy',
    help="Select the game mode."
)
args = parser.parse_args()

print(f"Game mode selected: {args.mode}")
```

Running with invalid choice

```
$ python script.py --mode expert
Output: error: argument
--mode: invalid choice:
'expert' (choose from
'easy', 'medium', 'hard')
```



Making Optional Arguments Required

Sometimes an argument has a flag (-) but is still mandatory. Use `required=True`.

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument(
    "-o", "--output",
    required=True,
    help="Specify the output file
    name (required).")
)
args = parser.parse_args()

print(f"Output will be saved to {
    args.output}")
```

Running without required argument

```
$ python script.py
Output: error: the
following arguments are
required: -o/--output
```



Variable Number of Arguments with `nargs`

`nargs` determines how many arguments a single action consumes.

- `'?'`: One argument, which can be optional.
- `'*'`: Zero or more arguments (produces a list).
- `'+'`: One or more arguments (produces a list).



Question: Behavior of nargs='+'

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument(
    'filenames',
    nargs='+', # one or more
    help='List of files to process'
)
args = parser.parse_args()

print("Processing the following
      files:")
for filename in args.filenames:
    print(f"- {filename}")
```

Command 1

```
$ python script.py
report.txt data.csv
```

Command 2

```
$ python script.py
```



Answer: Behavior of nargs='+'

`nargs='+'` consumes all available positional arguments into a list.

Processing the following files:

- report.txt
- data.csv

`nargs='+'` requires at least one argument.

```
usage: script.py [-h] filenames [filenames ...]
script.py: error: the following arguments are required:
  filenames
```



Mutually Exclusive Arguments

For arguments that cannot be used together (e.g., `--verbose` and `--quiet`).

```
import argparse
parser = argparse.ArgumentParser()
group = parser.add_mutually_exclusive_group()
group.add_argument("-v", "--verbose", action="store_true")
group.add_argument("-q", "--quiet", action="store_true")
args = parser.parse_args()
if args.verbose:
    print("Verbose output")
```

```
elif args.quiet:
    print("Quiet output")
else:
    print("Normal output")
```

Running with both flags

```
$ python script.py -v -q
Output: error: argument
-q/--quiet: not allowed
with argument -v/--verbose
```



Sub-parsers for Commands

Useful for creating tools with multiple actions, like `git clone` or `git push`.

```
import argparse
parser = argparse.ArgumentParser()
subparsers = parser.add_subparsers(dest='command', help='sub-
    command help')
```



Sub-parsers for Commands

```
# Create the parser for the "clone" command
parser_clone = subparsers.add_parser('clone', help='clone a
    repository')
parser_clone.add_argument('url', help='the repository URL')

# Create the parser for the "push" command
parser_push = subparsers.add_parser('push', help='push to a
    repository')
parser_push.add_argument('remote', help='the remote name')
```



Sub-parsers for Commands

```
args = parser.parse_args()

if args.command == 'clone':
    print(f'Cloning repository from {args.url}')
elif args.command == 'push':
    print(f'Pushing to remote {args.remote}')
```



Running Sub-parser Commands

Each sub-command has its own set of arguments and help messages.

Running the 'clone' command

```
$ python script.py clone https://github.com/user/repo.git  
Cloning repository from https://github.com/user/repo.git
```

Running the 'push' command

```
$ python script.py push origin  
Pushing to remote origin
```



Making Sub-commands Required

By default, a sub-command is not required. Running the script without one does nothing.

Problem

```
$ python script.py
```

(No output, no error)

To fix this, use **required=True** (Python 3.7+).

```
# In the original setup:  
subparsers = parser.add_subparsers(  
    dest='command',  
    help='sub-command help',  
    required=True # Make the command mandatory  
)
```



Making Sub-commands Required

Running without a command (now fixed)

```
1 $ python script.py
2 usage: script.py [-h] {clone,push} ...
3 script.py: error: the following arguments are required: command
```



Questions

```
import argparse
def build_parser():
    p = argparse.ArgumentParser(prog="imgtool",
                                description="Manipulate images")
    sub = p.add_subparsers(dest="cmd", required=True)

    r = sub.add_parser("resize", help="resize images")
    r.add_argument("input", nargs="+")
    r.add_argument("--width", type=int, required=True)
    r.add_argument("--height", type=int, required=True)
    r.add_argument("--out", default="out/")

    c = sub.add_parser("convert", help="convert format")
    c.add_argument("input", nargs="+")
    c.add_argument("--format", choices=["png", "jpg", "webp"], required=True)
    return p
```



Answer

```
def cli():  
    args = build_parser().parse_args()  
    if args.cmd == "resize":  
        ...  
    elif args.cmd == "convert":  
        ...  
  
if __name__ == "__main__":  
    cli()
```



Python Scripting

Thank You for Listening!